

Caching Strategy Part 3

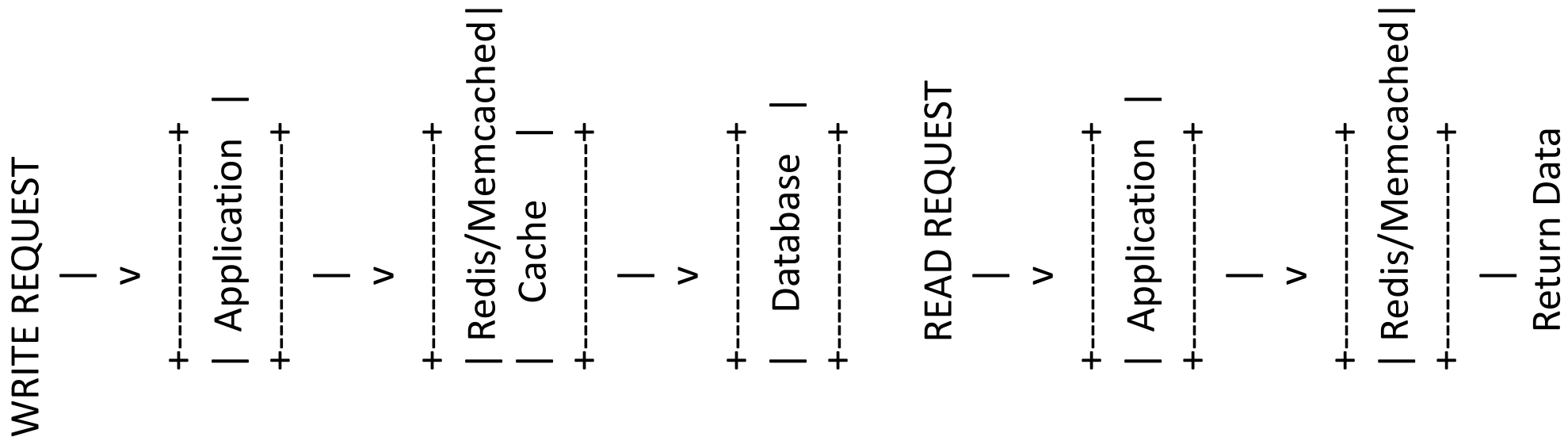
Prepared By: Dr. D. P. Singh

Writing Strategies

- Write-Through
- Write-Back
- Write-Around

Write-Through Overview

- **Write-through** caching strategy involves writing data to **both the cache and the primary database** simultaneously.
- The application writes data to the **cache first, and the cache immediately updates the database.**

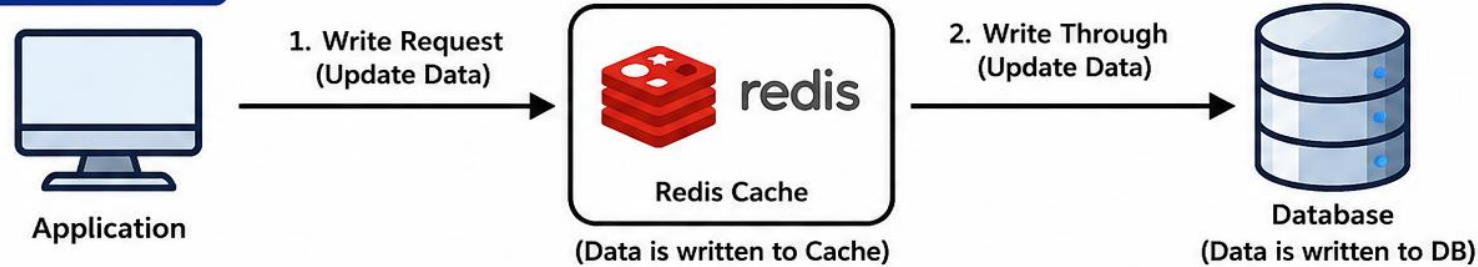


Working

Write-Through Caching with **Redis** (or **Memcached**)

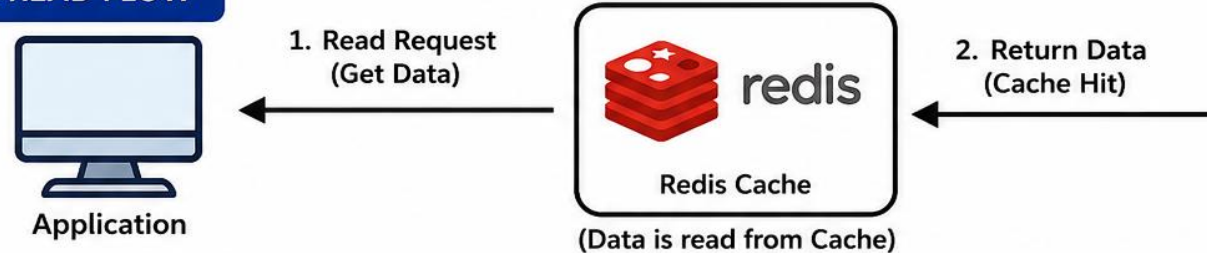
In Write-Through, every write goes to the cache first, and then to the database.
So both cache and database stay in sync.

1. WRITE FLOW



Result:
Data is written to both Redis and Database synchronously.

2. READ FLOW



Result:
Data is returned from Redis.
Database is not hit on read.

Key Points

- Write goes to Cache (Redis/Memcached) first
- Then Cache writes to Database
- Keeps Cache and Database consistent
- Reads are fast (served from Cache)
- Slightly higher write latency

Example (User Update)

User updates name = "John"

1. Application → **Redis SET user:1:name "John"**
2. Redis → **UPDATE users SET name = 'John' WHERE id = 1;**

Now both Redis and Database have the latest value.

Advantages

- ✓ **Strong cache consistency**

Cache and database remain synchronized.

- ✓ **Fast reads**

Data is already available in cache.

- ✓ **No stale data after writes**

Users see updated information immediately.

- ✓ **Simple read logic**

Most reads come directly from Redis/Memcached.

Disadvantages

✗ Higher write latency

A write is not complete until both cache and database are updated.

✗ Cache stores unused data

Data is cached even if it is never read again.

✗ More resource usage

Every write operation touches two storage systems.

✗ Failure handling complexity

If database write fails after cache update, consistency problems can occur.

When to Use Write-Through

Use write-through when:

- **Read** operations are much **more frequent than writes**.
- Data consistency is important.
- User profiles, product catalogs, account settings, and configuration data are frequently read after being updated.
- **Typical Use Cases**
 - E-commerce product information
 - User profiles
 - Banking account details
 - Session management
 - Inventory systems

Cache-Aside vs Write-Through

Feature	Cache-Aside	Write-Through
Read Speed	Fast after cache hit	Always fast
Write Speed	Faster	Slower
Consistency	Can become stale	Stronger consistency
Cache Population	On-demand	Automatic
Storage Usage	Lower	Higher

In Redis-based systems, **Cache-Aside** is the most common pattern, while **Write-Through** is chosen when keeping cache and database synchronized is more important than maximizing write performance.

- Thank You